

Parallel and Distributed Computing (CS-3216)

Lecture 03-04 (Physical Organization of Parallel Platforms)

Waqas Ali

Agenda

- 1 Flynn's Taxonomy
- 2 Interconnection Networks of Parallel Computers
- 3 Reference Books

Agenda

- 1 Flynn's Taxonomy
- 2 Interconnection Networks of Parallel Computers
- 3 Reference Books

Flynn's Taxonomy

It classifies architectures into four different types according to how instructions and data flow through cores

- Single Instruction Single Data (SISD)
- Single Instruction Multiple Data (SIMD)
- Multiple Instruction Single Data (MISD)
- Multiple Instruction Multiple Data (MIMD)

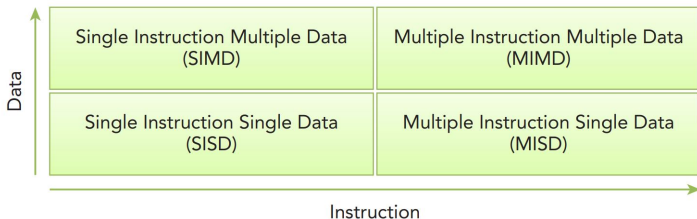


Figure: Architectures of Flynn

Single Instruction Single Data

- SISD is the simplest category and represents traditional sequential processing.
- It consists of a single processing unit (CPU) that operates on a single data stream at a time, executing one instruction at a time.
- Examples of SISD architectures include most traditional desktop computers and early mainframe computers.

Single Instruction Multiple Data

- SIMD architectures execute the same instruction on multiple data elements simultaneously.
- They have a single control unit that fetches and decodes instructions, and multiple processing units that operate in parallel.
- Each processing unit performs the same operation on different data elements, achieving data-level parallelism. SIMD architectures are suitable for tasks that involve large amounts of data parallelism, such as multimedia processing and vector calculations.
- Examples of SIMD architectures include graphics processing units (GPUs) and vector processing units.

Multiple Instruction Single Data

- MISD architectures are rare and less commonly used. They involve multiple processing units that operate on the same data stream, but each processing unit executes a different instruction on the data.
- MISD architectures are primarily theoretical and have limited practical applications.
- Examples of MISD architectures are not commonly found in real-world computing systems.

Multiple Instruction Multiple Data

- MIMD architectures consist of multiple processing units that can independently execute different instructions on different data streams.
- Each processing unit can operate asynchronously and execute its own instructions and access its own data. MIMD architectures offer the highest degree of parallelism and are commonly used in modern parallel computing systems.
- Examples of MIMD architectures include multi-core processors, distributed computing systems, and cluster-based supercomputers.

Agenda

- 1 Flynn's Taxonomy
- 2 Interconnection Networks of Parallel Computers
- 3 Reference Books

Static vs Dynamic Interconnections

- Interconnection networks carry data between processors and to memory
- Interconnects are made of processing elements, switches and links (wires)
- Interconnects are classified as static or dynamic
- **Static** networks consist of point-to-point communication links among processing nodes and are also referred to as direct networks
- **Dynamic** networks are built using switches and communication links. Dynamic networks are also referred to as indirect networks

Static vs Dynamic Interconnections

- Interconnection networks carry data between processors and to memory
- Interconnects are made of processing elements, switches and links (wires)
- Interconnects are classified as static or dynamic
- **Static** networks consist of point-to-point communication links among processing nodes and are also referred to as direct networks
- **Dynamic** networks are built using switches and communication links. Dynamic networks are also referred to as indirect networks

Static vs Dynamic Interconnections

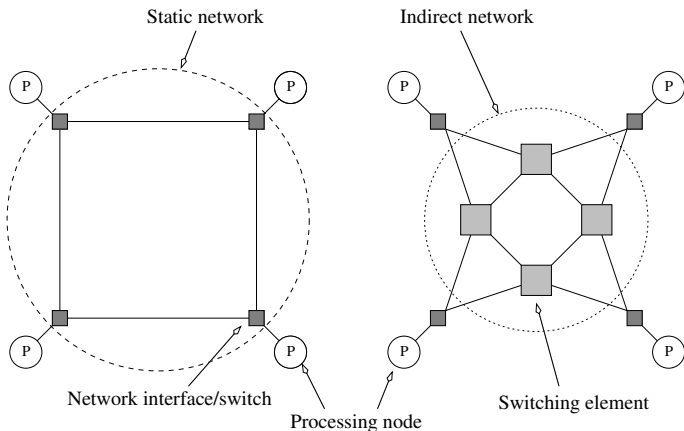


Figure: Classification of interconnection networks: (a) a static network; and (b) a dynamic network.

Network Topologies

- Variety of network topologies have been proposed and implemented
- These topologies tradeoff performance for cost
- Commercial machines often implement hybrids of multiple topologies for reasons of packaging, cost, and available components

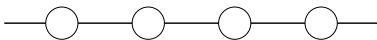
Network Topologies: Linear Arrays/Buses

- Some of the simplest and earliest parallel machines used buses
- All processors access a common bus for exchanging data
- In a linear array, each node has two neighbors, one to its left and one to its right
- If the nodes at either end are connected, we refer it as 1-D torus or a ring
- The distance between any two nodes is $O(1)$ in a bus
- The bus also provides a convenient broadcast media
- However, the bandwidth of the shared bus is a major bottleneck
- Typical bus based machines are limited to dozens of nodes
- Sun Enterprise servers and Intel Pentium based shared-bus multiprocessors are examples of such architectures

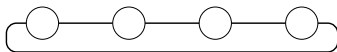
Network Topologies: Linear Arrays/Buses

- Some of the simplest and earliest parallel machines used buses
- All processors access a common bus for exchanging data
- In a linear array, each node has two neighbors, one to its left and one to its right
- If the nodes at either end are connected, we refer it as 1-D torus or a ring
- The distance between any two nodes is $O(1)$ in a bus
- The bus also provides a convenient broadcast media
- However, the bandwidth of the shared bus is a major bottleneck
- Typical bus based machines are limited to dozens of nodes
- Sun Enterprise servers and Intel Pentium based shared-bus multiprocessors are examples of such architectures

Linear Array, Ring



(a)



(b)

Figure: Linear arrays: (a) with no wraparound links; (b) with wraparound link.

Evaluating Static Interconnections

The parameters to evaluate a static interconnection:-

- **Cost:** Usually depends on number of links for communication. E.g., cost for bus is $p - 1$
- **Diameter:** The shortest distance between the two farthest nodes in a network. The diameter of a linear array is $p - 1$
- **Bisection Width:** The minimum number of links you must cut to divide the network into two [Nearly] equal parts. The bisection width of a linear array is 1

Evaluating Static Interconnections

The parameters to evaluate a static interconnection:-

- **Cost:** Usually depends on number of links for communication. E.g., cost for bus is $p - 1$
 - Lower values are favorable
- **Diameter:** The shortest distance between the two farthest nodes in a network. The diameter of a linear array is $p - 1$
- **Bisection Width:** The minimum number of links you must cut to divide the network into two [Nearly] equal parts. The bisection width of a linear array is 1

Evaluating Static Interconnections

The parameters to evaluate a static interconnection:-

- **Cost:** Usually depends on number of links for communication. E.g., cost for bus is $p - 1$
- **Diameter:** The shortest distance between the two farthest nodes in a network. The diameter of a linear array is $p - 1$
- **Bisection Width:** The minimum number of links you must cut to divide the network into two [Nearly] equal parts. The bisection width of a linear array is 1

Evaluating Static Interconnections

The parameters to evaluate a static interconnection:-

- **Cost:** Usually depends on number of links for communication. E.g., cost for bus is $p - 1$
- **Diameter:** The shortest distance between the two farthest nodes in a network. The diameter of a linear array is $p - 1$
 - Lower values are favorable
- **Bisection Width:** The minimum number of links you must cut to divide the network into two [Nearly] equal parts. The bisection width of a linear array is 1

Evaluating Static Interconnections

The parameters to evaluate a static interconnection:-

- **Cost:** Usually depends on number of links for communication. E.g., cost for bus is $p - 1$
- **Diameter:** The shortest distance between the two farthest nodes in a network. The diameter of a linear array is $p - 1$
- **Bisection Width:** The minimum number of links you must cut to divide the network into two [Nearly] equal parts. The bisection width of a linear array is 1

Evaluating Static Interconnections

The parameters to evaluate a static interconnection:-

- **Cost:** Usually depends on number of links for communication. E.g., cost for bus is $p - 1$
- **Diameter:** The shortest distance between the two farthest nodes in a network. The diameter of a linear array is $p - 1$
- **Bisection Width:** The minimum number of links you must cut to divide the network into two [Nearly] equal parts. The bisection width of a linear array is 1
 - What it tells about performance of a topology?
 - Assigned Reading: bisection bandwidth

Evaluating Static Interconnections

The parameters to evaluate a static interconnection:-

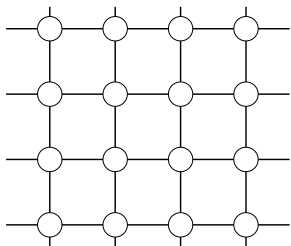
- **Arc-connectivity:** The minimum number of arcs or links that must be removed from the network, to break the network into two disconnected networks

Evaluating Static Interconnections

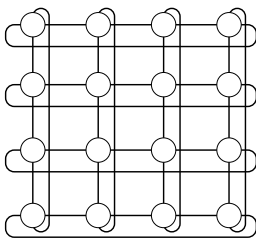
The parameters to evaluate a static interconnection:-

- **Arc-connectivity:** The minimum number of arcs or links that must be removed from the network, to break the network into two disconnected networks
 - Higher value are desirable
 - It is minimum number of the links that must be cut to separate the single node from the network
 - Higher values means, that in case of link failure there are multiple other routes to the node

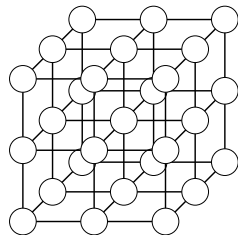
Meshes and $k - D$ meshes



(a)



(b)



(c)

Figure: Two and three dimensional meshes: (a) 2-D mesh with no wraparound; (b) 2-D mesh with wraparound link (2-D torus); and (c) a 3-D mesh with no wraparound.

Meshes and $k - D$ meshes

Mesh

- In a linear array, each node has two neighbors, one to its left and one to its right. If the nodes at either end are connected, we refer to it as a 1-D torus or a ring.
- A generalization to 2 dimensions where each node has 4 neighbors [except border nodes]
- A further generalization to d dimensions has nodes with $2d$ neighbors in each dimension (i.e., 6 neighbors in case of 3d cube)

Completely Connected and Star Connected Networks

- In completely connected network, Each processor is connected to every other processor.
- The number of links in the network scales as $O(p^2)$.
- While the performance scales very well, the hardware complexity is not realizable for large values of p

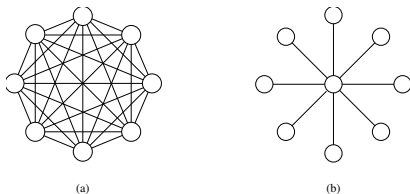


Figure: (a) A completely-connected network of eight nodes; (b) a Star connected network of nine nodes.

Completely Connected and Star Connected Networks

- In Star network, Every node is connected only to a common node at the center.
- Distance between any pair of nodes is $O(1)$. However, the central node becomes a bottleneck.
- In this sense, star connected networks are static counterparts of buses

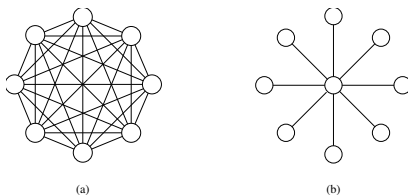


Figure: (a) A completely-connected network of eight nodes; (b) a Star connected network of nine nodes.

Hypercubes and their Construction

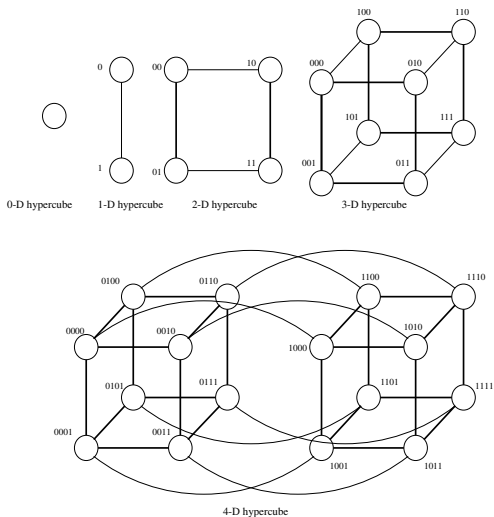


Figure: Construction of hypercubes from hypercubes of lower dimension.

Hypercubes and their Construction

- The shortest-distance between any two nodes is at most $\log(p)$.
- $d = \log(p)$ (*dimensions* = $\log(\text{nodes})$)
- Each node has $\log(p)$ neighbors.
- The distance between two nodes is given by the number of bit positions at which the two nodes differ

Tree-Based Networks

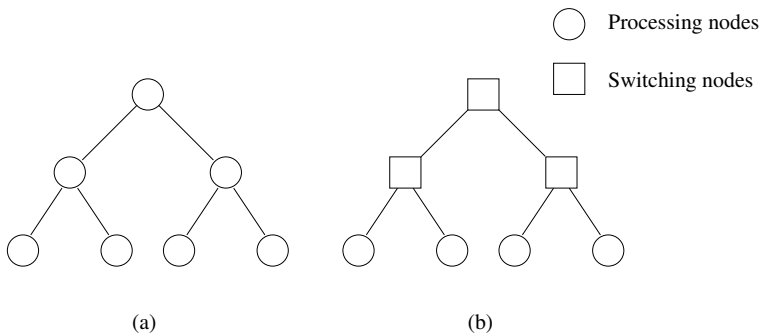


Figure: Complete binary tree networks: (a) a static tree network; and (b) a dynamic tree network.

Tree-Based Networks

- A tree network is one in which there is one path between any pair of nodes
- Linear arrays and star-connected networks are special cases of tree-based networks
- In static tree network, each node represent a processing element
- In dynamic tree network, leaf nodes represent processing element while internal nodes are switching elements.
- The source node sends the message up the tree until it reaches the node at the root of the smallest subtree containing both the source and destination nodes

Tree-Based Networks

- A tree network is one in which there is one path between any pair of nodes
- Linear arrays and star-connected networks are special cases of tree-based networks
- In static tree network, each node represent a processing element
- In dynamic tree network, leaf nodes represent processing element while internal nodes are switching elements.
- The source node sends the message up the tree until it reaches the node at the root of the smallest subtree containing both the source and destination nodes

Tree-Based Networks

- A tree network is one in which there is one path between any pair of nodes
- Linear arrays and star-connected networks are special cases of tree-based networks
- In static tree network, each node represent a processing element
- In dynamic tree network, leaf nodes represent processing element while internal nodes are switching elements.
- The source node sends the message up the tree until it reaches the node at the root of the smallest subtree containing both the source and destination nodes

Properties of Complete Binary Tree Network

- The distance between any two nodes is no more than $2 \log p$.
- Links higher up the tree potentially carry more traffic than those at the lower levels.
- For this reason, a variant called a fat-tree, fattens the links as we go up the tree.
- Trees can be laid out in 2D with no wire crossings. This is an attractive property of trees.

Properties of Complete Binary Tree Network

- The distance between any two nodes is no more than $2 \log p$.
- Links higher up the tree potentially carry more traffic than those at the lower levels.
- For this reason, a variant called a fat-tree, fattens the links as we go up the tree.
- Trees can be laid out in 2D with no wire crossings. This is an attractive property of trees.

Tree-Based Networks

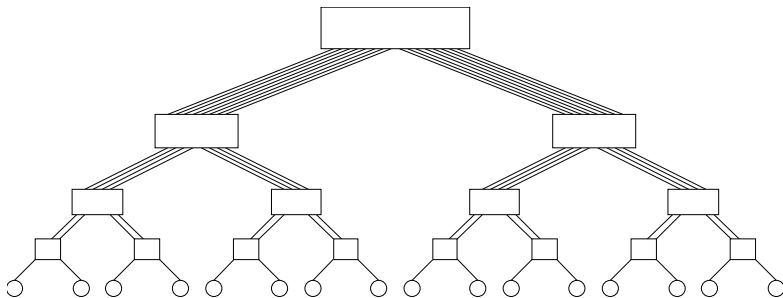


Figure: A fat tree network of 16 processing nodes.

Evaluating Static Interconnection Networks

Network	Diameter	Bisection Width	Arc Connectivity	Cost (No. of links)
Completely-connected	1	$\frac{p^2}{4}$	$p - 1$	$\frac{p(p-1)}{2}$
Star	2	1	1	$p - 1$
Complete binary tree	$2 \log(\frac{p+1}{2})$	1	1	$p - 1$
Linear array	$p - 1$	1	1	$p - 1$
2-D mesh, no wraparound	$2(\sqrt{p} - 1)$	$\sqrt{p}$	2	$2(p - \sqrt{p})$
2-D wraparound mesh	$2\lfloor \frac{\sqrt{p}}{2} \rfloor$	$2\sqrt{p}$	4	$2p$
Hypercube	$\log p$	$\frac{p}{2}$	$\log p$	$\frac{p \log p}{2}$

Agenda

- 1 Flynn's Taxonomy
- 2 Interconnection Networks of Parallel Computers
- 3 Reference Books

Reference Books

- **Introduction to Parallel Computing, Second Edition, by Ananth Grama**
- Parallel programming in C with MPI and OpenMP by Michael J. Quinn
- An Introduction to Parallel Programming by Peter S. Pacheco
- Professional CUDA C Programming by John Cheng